Compiler Structure: Middle-End & Back-End

Middle-End Compiler Structure

- The middle-end focuses on optimization.
- Goals: Improve execution time and reduce memory usage.
- It works on the Intermediate Form (IF) of the code.

- Code optimization is the process of improving the intermediate representation (IR) of the program to make the final generated code faster, smaller, and more efficient, without changing its output or meaning.
- It is performed in the middle-end of the compiler, *after semantic analysis* and *before code generation*.

Purpose/Goal of Optimization

- Main objectives:
- - Faster execution:Reduce execution time
- - Reduced memory usage:Minimize memory usage
- Reduce number of instructions
- - Efficient CPU resource utilization
- Improve CPU resource utilization
- Increase cache friendliness and pipeline efficiency
- Optimization happens after semantic analysis and before code generation.

Type	Description	Example
Constant Folding	Compile-time evaluation of expressions	x = 3 + 5; → x = 8
Dead Code Elimination	Removes code that never affects output	if (false) { ... } is removed
Strength Reduction	Replace expensive ops with cheaper ones	x = y * 2; → x = y + y;
Loop Unrolling	Reduce loop overhead	for(i=0;i<2;i++) sum += a[i]; → sum += a[0]; sum += a[1];
Common Subexpression Elimination	Reuse repeated expressions	x = (a+b)*(a+b); → t = a+b; x = t*t;
Inlining Functions	Replace function calls with body	foo(); → actual code of foo inserted

Example Before & After Optimization:

- Before Optimization:
- int x = 3 * 4;
- int y = x + 0;
- int z = y + 0;
- After Optimization:
- int x = 12;
- int y = x;
- int z = y;

What happened:

- 3 * 4 → 12 (Constant Folding)
- + 0 → removed (Identity Operation Elimination)

Intermediate Form (IF)

- Generated after semantic analysis
- Machine-independent representation of code
- Enables portability and ease of optimization
- Example: 3-address code, abstract syntax trees

Managed Code Example

- C#, VB.NET and other .NET languages compile to Intermediate Language (IL)
- IL runs under the CLR (Common Language Runtime)
- Allows interoperability and portability across .NET-supported platforms

Back-End Compiler Structure

- Final phase of compilation: Code Generation
- Transforms Intermediate Form to Target Code
- Uses symbol table and runtime environment for conversion
- The Back-End of the compiler is the final phase of the compilation process.
- It takes the optimized Intermediate Representation (IR) and translates it into target machine code (or bytecode in hybrid systems like Java).

Major Responsibilities of the Back-End:

- Code Generation
- Register Allocation
- Instruction Selection & Scheduling
- Machine-Dependent Optimization
- Handling of Symbol Tables
- Final Code Emission

Back-End Phases Breakdown:

Phase	Description
Instruction Selection	Converts IR to specific machine instructions.
Register Allocation	Assigns variables to a limited number of CPU registers.
Instruction Scheduling	Reorders instructions to avoid pipeline stalls or delays.
Code Emission	Produces the actual target language code (binary/assembly).

Compilation of a Simple C Statement

- Code:
- int x = a + b * c;
- Intermediate Representation (IR - 3-address code):
- t1 = b * c
- x = a + t1

- Back-End Code Generation (x86-like pseudo assembly):
- MOV R1, b
- MUL R1, c ; R1 = b * c
- MOV R2, a
- ADD R2, R1 ; R2 = a + (b*c)
- MOV x, R2 ; store result into x

Role of the Symbol Table in the Back-End

- Maps variable names (a, b, c, x) to memory locations or registers.

- Tracks scope, data type, and usage count (for register allocation and debugging).

Back-End in Hybrid Systems

- Example: Java Compilation
- Java source → Bytecode (IR)
- Bytecode is not immediately machine code.
- Java Virtual Machine (JVM) uses Just-In-Time (JIT) Compilation at runtime to convert bytecode to machine code.
- This makes Java platform-independent but efficient at runtime.

Output of Back-End

- For native compilers: Machine language (e.g., .exe, .o, .out)
- For hybrid compilers: Portable bytecode (e.g., .class in Java, .dll or .exe for .NET)

What is Pure Interpretation?

- Pure Interpretation is a method of executing a program directly without translating it into machine code. The source code is executed line-by-line or statement-by-statement using an interpreter at runtime.
- In contrast to compilation (which translates the whole code beforehand), pure interpretation reads and executes code on the fly.

Key Characteristics of Pure Interpretation:

Feature	Description
Line-by-line Execution	Executes one statement at a time.
No Target Code	Does not produce intermediate machine code or binary.
Slower Execution	More time-consuming than compiled execution.
High Flexibility	Easy for debugging, testing, and educational tools.
Memory-Efficient	No need to store binary or executable files.

Example: Pure Interpretation in Python

- a = 5
- b = 3
- print(a + b)
- **Interpretation Process:**
- Interpreter reads a = 5 → Stores a in memory.

- Reads b = 3 → Stores b in memory.

- Reads print(a + b) → Fetches values from memory → Computes 8 → Outputs result.

- No translation to machine code, only runtime execution.

Languages That Use Pure Interpretation:

Language	Interpreter-Based?
Python	Yes
JavaScript	Yes
Ruby	Yes
Lisp	Traditionally interpreted
BASIC	Early versions used pure interpretation

Limitations of Pure Interpretation:

- Slower performance due to repeated parsing.
- Less suitable for performance-critical applications.
- Can’t optimize like a compiler.

Advantages of Pure Interpretation:

- Simpler implementation.
- Better suited for scripting, rapid prototyping, and education.
- Immediate feedback and easier debugging.

What is Hybrid Implementation?

- Hybrid Implementation is a combination of both compilation and interpretation. In this approach:
- The source code is first compiled into an intermediate form (not machine code).
- This intermediate code is then interpreted or executed by a virtual machine at runtime.

Key Characteristics of Hybrid Implementation:

Feature	Description
Two-Stage Process	Compilation + Interpretation
Platform Independence	Intermediate code runs on any system with a suitable virtual machine
Faster than pure interpretation	Because the source is precompiled
More flexible than compilation	Easier to manage across different platforms

Example: Java Hybrid Model

- Step 1: Compilation

- Java source code (.java) is compiled into Bytecode (.class file) using the Java Compiler (javac).
- Step 2: Interpretation

- The Java Virtual Machine (JVM) then interprets the bytecode at runtime or uses Just-In-Time (JIT) Compilation to convert it to native machine code during execution.

- This allows the same .class file to run on any OS with a JVM installed.

Hybrid Compilation Example

- Java code → Compiled to Java Bytecode → Runs on JVM
- .NET code → Compiled to IL → Runs on CLR
- Machine code generation deferred to runtime (JIT - Just-In-Time Compilation)

Symbol Table Usage

- Maintains info about identifiers (variables, functions, types)
- Used during optimization and code generation
- May be retained by the debugger post-compilation

Platform-Independent Code Generation

- Generates code runnable on multiple platforms
- Example: Java bytecode can run on Windows, Linux, Android
- Intermediate languages enable Write Once, Run Anywhere

Conclusion

- Middle-End: Optimizes Intermediate Form
- Back-End: Generates platform-specific code using optimized IR and symbol table
- Hybrid models defer machine code generation to runtime